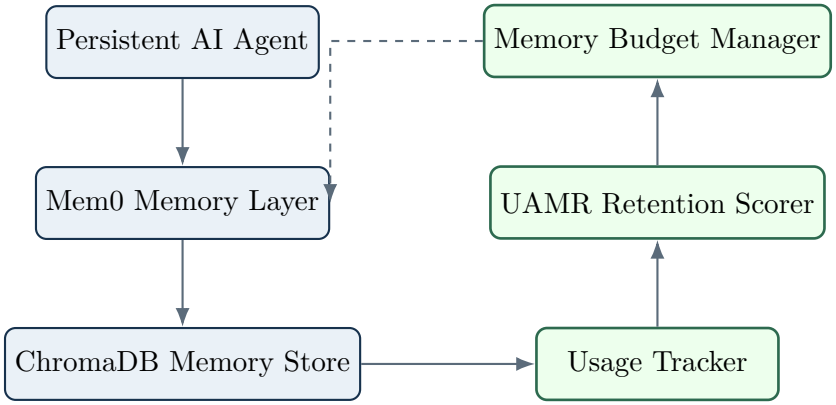
Usage-Aware Memory Retention for Persistent AI Agents

A Lightweight Retention Policy Implemented on Top of Mem0



Final-Year Project - Research Methodology Document

Prepared for project review and presentation

1 Problem Statement

Long-term AI agents require persistent memory to preserve information across conversations and tasks. A memory framework such as Mem0 can extract, store, update, and retrieve useful information for later interactions. However, when an agent operates over long periods, the number of stored memories can grow continuously. Retaining all historical information creates a larger search space, increases storage and retrieval overhead, introduces semantically redundant memories, and can add irrelevant context to the prompt supplied to the language model.

At the opposite extreme, aggressive forgetting can remove information that is still important for future reasoning. Therefore, the core problem is not simply how to store memory, but how to decide which memories should remain actively available when the memory capacity is constrained.

Research question: Under an equal active-memory budget, can a usage-aware retention policy preserve long-term task performance better than simpler retention strategies such as random, recency-only, frequency-only, and relevance-only retention?

The project proposes a lightweight Usage-Aware Memory Retention (UAMR) policy that assigns each memory a retention score derived from four observable signals: retrieval frequency, recency, historical retrieval quality, and semantic redundancy. Memories are ranked using this score, and only the highest-value memories remain active under a fixed memory budget.

Core Design Principle

The project does not attempt to replace Mem0 or introduce a new end-to-end memory architecture. Mem0 remains the baseline memory framework. The research contribution is a modular retention policy that operates on memories already created by Mem0 and uses post-deployment usage evidence to decide which memories should remain active.

2 Objectives

The project has five concrete objectives:

1. **Build a persistent memory-enabled AI agent.** Use an LLM with Mem0 and ChromaDB to establish a working baseline capable of storing and retrieving long-term memories.
2. **Instrument memory usage.** Track how often each memory is retrieved, when it was last retrieved, its historical retrieval rank, and semantic similarity to the queries for which it was returned.
3. **Design the UAMR scoring function.** Compute a mathematically defined retention score using retrieval frequency, recency, retrieval quality, and redundancy.
4. **Operate under controlled memory budgets.** Evaluate behavior when the active memory store is restricted to 100%, 75%, 50%, and 25% of the memories produced by the baseline system.
5. **Evaluate the proposed policy scientifically.** Compare UAMR against random, recency, frequency, and relevance baselines using long-term-memory benchmarks and metrics such as

QA performance, Recall@K, active memory count, redundancy rate, context tokens, and retrieval latency.

3 System Methodology and Architecture

3.1 Baseline System

The baseline system is a persistent AI agent that communicates with an Ollama-hosted language model and uses Mem0 for long-term memory management. ChromaDB stores the memory text, embeddings, and associated vector-search information.

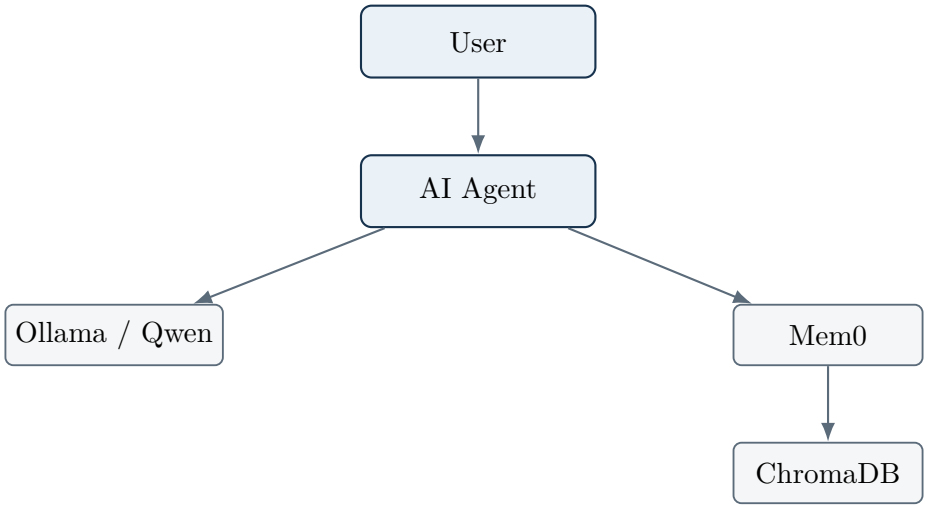


Figure 1: Baseline persistent-agent architecture.

3.2 Proposed Research Layer

The proposed system adds three modules without modifying the agent's core behavior:

- **Usage Tracker:** records retrieval events and historical use statistics for every memory.
- **Retention Scorer:** computes the four signal values and the final UAMR score.
- **Memory Budget Manager:** ranks memories by score and marks them ACTIVE or PRUNED for a specified budget. During experiments, memories are not physically deleted.

4 Mathematical Formulation

Let the memory store be

$$M = \{m_1, m_2, \ldots, m_N\},$$

and let $B < N$ be the maximum number of memories that may remain active. The ideal objective is to choose a subset A such that

$$A^* = \arg \max_{A \subseteq M,\, |A| \leq B} \mathbb{E}[\text{future task performance} \mid A].$$

Future task utility cannot be observed directly at retention time. Therefore, UAMR estimates the likely future value of each memory from observable historical usage signals.

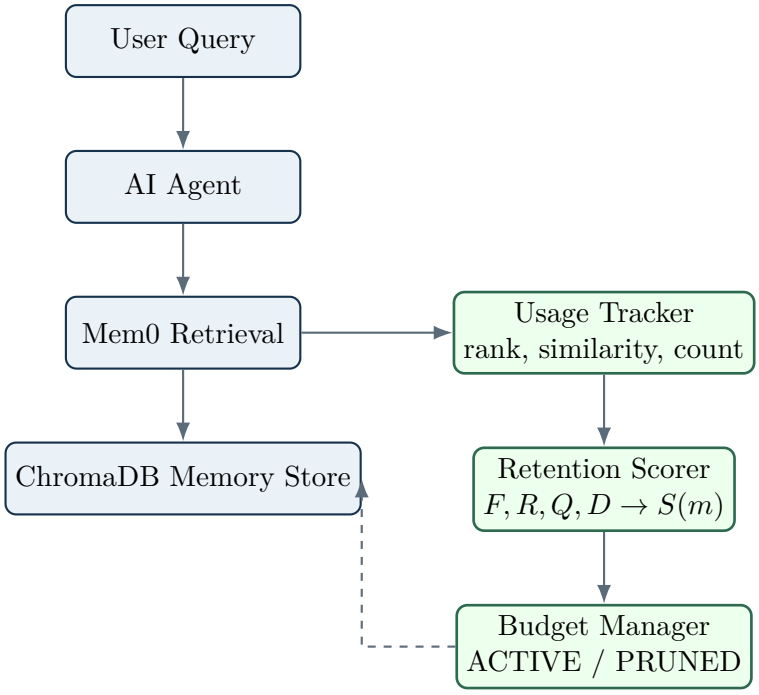


Figure 2: Proposed UAMR instrumentation and retention pipeline.

4.1 Retrieval Frequency $F(m)$

Let c_m be the number of times memory m has been retrieved and n_m the number of interactions since the memory was created. A Bayesian-smoothed retrieval-frequency estimate is

$$\boxed{F(m) = \frac{c_m + \alpha}{n_m + \alpha + \beta}}$$

with default $\alpha = \beta = 1$. The smoothing avoids zero-valued probabilities for new memories and prevents raw counts from unfairly favoring old memories.

4.2 Recency $R(m)$

Let Δ_m be the number of interactions since memory m was last retrieved. Recency is represented using exponential decay:

$$\boxed{R(m) = e^{-\Delta_m/\tau}}$$

where τ is a configurable decay parameter. Interaction count is preferred over wall-clock time because it produces deterministic, reproducible benchmark runs.

4.3 Historical Retrieval Quality $Q(m)$

Whenever memory m is retrieved for query q, let $s_{m,q}$ be its semantic similarity and $r_{m,q}$ its retrieval rank. A rank-discounted retrieval contribution is

$$q_{m,q} = \frac{s_{m,q}}{\log_2(r_{m,q} + 1)}.$$

The historical retrieval-quality score is

$$\boxed{Q(m) = \frac{1}{c_m} \sum_{q \in \text{Retrievals}(m)} \frac{s_{m,q}}{\log_2(r_{m,q} + 1)}}$$

with $Q(m) = 0$ if the memory has never been retrieved. This distinguishes consistently high-ranked memories from memories that repeatedly appear near the bottom of a top-k result set.

4.4 Semantic Redundancy $D(m)$

A preliminary value is first computed without redundancy:

$$V(m) = w_F F(m) + w_R R(m) + w_Q Q(m).$$

Let e_i be the embedding of memory m_i. A lower-value memory is penalized only when a higher-value memory is semantically very similar. Define

$$D(m_i) = \max_{\substack{j \neq i \\ V(m_j) > V(m_i)}} g(\cos(e_i, e_j)),$$

where

$$g(s) = \begin{cases} 0, & s < \theta_d, \\ \dfrac{s - \theta_d}{1 - \theta_d}, & s \geq \theta_d. \end{cases}$$

The asymmetry is important: the weaker duplicate is penalized, while the stronger representative is not automatically penalized merely for having a similar neighbor.

5 Final Retention Function

The UAMR retention score is

$$\boxed{S(m) = w_F F(m) + w_R R(m) + w_Q Q(m) - \lambda D(m)}.$$

The weights w_F, w_R, w_Q and redundancy penalty λ are treated as hyperparameters. A neutral initial configuration is

$$w_F = w_R = w_Q = \frac{1}{3}, \qquad \lambda = 0.25,$$

but these values are not claimed to be optimal. They should be selected using validation data and frozen before final test evaluation.

5.1 Cold-Start Protection

A newly created memory may have zero retrieval history despite being important. To prevent premature removal, define a grace period G. If

$$\text{age}(m) < G,$$

then memory m is protected and cannot be pruned during that retention cycle.

5.2 Memory Budget

For a budget fraction $b \in \{1.00, 0.75, 0.50, 0.25\}$, the active-memory capacity is

$$B = \lfloor bN \rfloor.$$

All eligible memories are ranked by $S(m)$. The highest-scoring memories are marked ACTIVE until the active-memory budget is reached; the remaining eligible memories are marked PRUNED. During development and experimentation, PRUNED memories remain stored but are excluded from active retrieval so that runs can be reproduced and reversed.

6 UAMR Algorithm

Algorithm 1 Usage-Aware Memory Retention (UAMR)

Require: Memory set M, budget B, parameters $w_F, w_R, w_Q, \lambda, \tau, \theta_d, G$
Ensure: Active memory subset A

for all $m \in M$ **do**
    Compute $F(m)$ using smoothed retrieval frequency
    Compute $R(m)$ using exponential recency decay
    Compute $Q(m)$ using rank-discounted retrieval similarity
    $V(m) \leftarrow w_F F(m) + w_R R(m) + w_Q Q(m)$
end for
for all $m_i \in M$ **do**
    Compute $D(m_i)$ against higher-value memories only
    $S(m_i) \leftarrow V(m_i) - \lambda D(m_i)$
    if age$(m_i) < G$ **then**
        Mark m_i as protected
    end if
end for
Add protected memories to A
Rank remaining eligible memories by $S(m)$ descending
Add highest-ranked memories until $|A| = B$ (or until all eligible memories are exhausted)
Mark memories in A as ACTIVE and remaining eligible memories as PRUNED
return A

7 Evaluation Methodology

A fair evaluation requires all retention strategies to operate on the same Mem0-generated memory set and under the same active-memory budget. The proposed UAMR policy should be compared with the following baselines:

Policy	Retention rule
Random	Select a random subset of exactly the allowed budget.
Recency	Rank memories using $R(m)$ only.
Frequency	Rank memories using $F(m)$ only.
Relevance	Rank memories using $Q(m)$ only.
UAMR	Rank using $w_F F + w_R R + w_Q Q - \lambda D$.

7.1 Primary Benchmark

LoCoMo is suitable as the primary long-term-memory benchmark because it contains long, multi-session conversations followed by questions that require information from earlier interactions. The experimental workflow is:

7.2 Metrics

The primary analysis should report both task quality and efficiency rather than a single aggregate score:

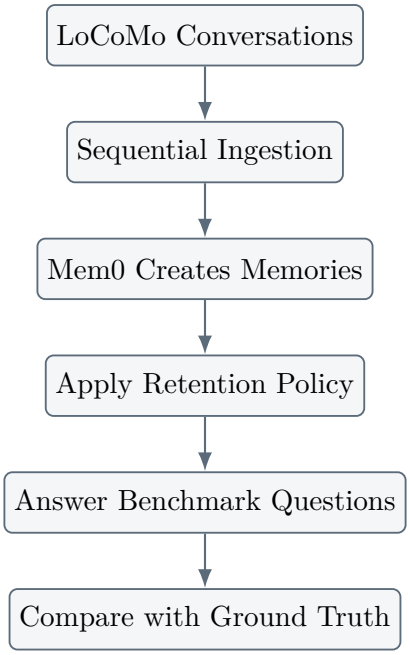


Figure 3: Benchmark evaluation pipeline.

- **QA Accuracy / F1:** downstream answer quality.
- **Recall@5 and Recall@10:** whether evidence memories required for answering are still retrieved.
- **Active Memory Count:** number of memories available under the selected budget.
- **Redundancy Rate:** proportion of active memories that are near-duplicates.
- **Retrieved Context Tokens:** memory-related tokens supplied to the LLM per query.
- **Retrieval Latency:** average time required to retrieve memory context.

8 Ablation Study

To determine which signals are responsible for performance, evaluate the full policy against variants with one component removed:

$$F + R + Q - D,$$

$$R + Q - D \quad \text{(without frequency)},$$

$$F + Q - D \quad \text{(without recency)},$$

$$F + R - D \quad \text{(without retrieval quality)},$$

$$F + R + Q \quad \text{(without redundancy)}.$$

This analysis is important because it explains *why* the proposed policy succeeds or fails instead of reporting only a final number.

9 Expected Result Interpretation

The primary result should be a performance-versus-memory-budget curve. A meaningful contribution does not require UAMR to increase full-memory accuracy. For example, if a 50% memory budget preserves nearly all of the full-memory QA score while reducing active memory, retrieval latency, context tokens, or redundancy, that demonstrates a useful performance-efficiency trade-off.

Core contribution: A lightweight, usage-aware retention policy that combines historical retrieval frequency, temporal recency, rank-aware retrieval quality, and semantic redundancy to prioritize persistent agent memories under fixed memory-capacity constraints.

10 Methodology Summary



Research hypothesis. Under an equal memory budget, UAMR should preserve downstream long-term memory performance more effectively than random, recency-only, frequency-only, and relevance-only retention if the combined usage signals provide a better estimate of future memory value.